

Concepts

Backtesting

Backtesting simulates trading using a specific system implementation. The system comprises the built-in engines, `Cache`, [MessageBus](#), `Portfolio`, [Actors](#), [Strategies](#), [Execution Algorithms](#), and user-defined modules. A `BacktestEngine` processes a stream of historical data. When the stream is exhausted, the engine produces results and performance metrics for analysis.

NautilusTrader offers two API levels for backtesting:

- **High-level API:** Uses a `BacktestNode` and configuration objects (`BacktestEngine`s are used internally).
- **Low-level API:** Uses a `BacktestEngine` directly with more "manual" setup.

Choosing an API level

Consider using the **low-level** API when:

- Your entire data stream can be processed within the available machine resources (e.g., RAM).
- You prefer not to store data in the Nautilus-specific Parquet format.
- You have a specific need or preference to retain raw data in its original format (e.g., CSV, binary, etc.).
- You require fine-grained control over the `BacktestEngine`, such as the ability to re-run backtests on identical datasets while swapping out components (e.g., actors or strategies) or adjusting parameter configurations.

Consider using the **high-level** API when:

- Your data stream exceeds available memory, requiring streaming data in batches.

- You want the performance and convenience of the `ParquetDataCatalog` for storing data in the Nautilus-specific Parquet format.
- You value the flexibility and functionality of passing configuration objects to define and manage multiple backtest runs across various engines simultaneously.

Low-level API

The low-level API centers around a `BacktestEngine`, where inputs are initialized and added manually via a Python script. An instantiated `BacktestEngine` can accept the following:

- Lists of `Data` objects, which are automatically sorted into monotonic order based on `ts_init`.
- Multiple venues, manually initialized.
- Multiple actors, manually initialized and added.
- Multiple execution algorithms, manually initialized and added.

This approach offers detailed control over the backtesting process, allowing you to manually configure each component.

Loading large datasets efficiently

When working with large amounts of data across multiple instruments, the way you load data can significantly impact performance.

The performance consideration

By default, `BacktestEngine.add_data()` sorts the entire data stream (existing data + newly added data) on each call when `sort=True` (the default). This means:

- First call with 1M bars: sorts 1M bars.
- Second call with 1M bars: sorts 2M bars.
- Third call with 1M bars: sorts 3M bars.
- And so on...

This repeated sorting of increasingly large datasets can become a bottleneck when loading data for multiple instruments.

Optimization strategies

Strategy 1: Defer sorting until the end (recommended for multiple instruments)

```
from nautilus_trader.backtest.engine import BacktestEngine

engine = BacktestEngine()

# Setup venue and instruments
engine.add_venue(...)
engine.add_instrument(instrument1)
engine.add_instrument(instrument2)
engine.add_instrument(instrument3)

# Load all data WITHOUT sorting on each call
engine.add_data(instrument1_bars, sort=False)
engine.add_data(instrument2_bars, sort=False)
engine.add_data(instrument3_bars, sort=False)

# Sort once at the end - much more efficient!
engine.sort_data()

# Now run your backtest
engine.add_strategy(strategy)
engine.run()
```



Strategy 2: Collect and add in a single batch

```
# Collect all data first
all_bars = []
all_bars.extend(instrument1_bars)
all_bars.extend(instrument2_bars)
all_bars.extend(instrument3_bars)

# Add once with sorting
engine.add_data(all_bars, sort=True)
```



Strategy 3: Use streaming API for very large datasets

For datasets that don't fit in memory, there are two streaming approaches:

Automatic chunking - supply a generator that yields batches. The engine pulls chunks lazily during a single `run()` call:

```
def data_generator():
    # Yield chunks of data (each chunk is a list of Data objects)
    yield load_chunk_1()
    yield load_chunk_2()
```



```

yield load_chunk_3()

engine.add_data_iterator(
    data_name="my_data_stream",
    generator=data_generator(),
)

engine.run() # Chunks are consumed on-demand

```

Manual chunking - load and run each batch yourself. This is the pattern used internally by `BacktestNode` and gives full control over batch boundaries:

```

engine.add_strategy(strategy)

for batch in data_batches:
    engine.add_data(batch)
    engine.run(streaming=True)
    engine.clear_data()

engine.end() # Finalize: flushes remaining timers, stops engines, produces results

```



i In streaming mode, timer advancement stops when data exhausts for each batch. Timers scheduled past the last data point (e.g. bar aggregation intervals) are deferred until more data arrives or `end()` is called, which flushes up to the `end` boundary from the last `run()` call.



Performance impact

For a backtest with 10 instruments, each with 1M bars:

- Sorting on each call: ~10 sorts of increasing size (1M, 2M, 3M, ... 10M bars).
- Sorting once at the end: 1 sort of 10M bars.

The deferred sorting approach can be **significantly faster** for large datasets.

Data loading contract

The `BacktestEngine` enforces important invariants to ensure data integrity:

Requirements:

- All data must be sorted before calling `run()`.
- When using `sort=False`, you **must** call `sort_data()` before running.
- The engine validates this and raises `RuntimeError` if unsorted data is detected.

- Calling `sort_data()` multiple times is safe (idempotent).

Safety guarantees:

- Data lists are always copied internally to prevent external mutations from affecting engine state.
- You can safely clear or modify data lists after passing them to `add_data()`.
- Adding data with `sort=True` makes it immediately available for backtesting.

This design ensures data integrity while enabling performance optimizations for large datasets.

High-level API

The high-level API centers around a `BacktestNode`, which orchestrates the management of multiple `BacktestEngine` instances, each defined by a `BacktestRunConfig`. Multiple configurations can be bundled into a list and processed by the node in one run.

Each `BacktestRunConfig` object consists of the following:

- A list of `BacktestDataConfig` objects.
- A list of `BacktestVenueConfig` objects.
- A list of `ImportableActorConfig` objects.
- A list of `ImportableStrategyConfig` objects.
- A list of `ImportableExecAlgorithmConfig` objects.
- An optional `ImportableControllerConfig` object.
- An optional `BacktestEngineConfig` object, with a default configuration if not specified.

Repeated runs

When conducting multiple backtest runs, it's important to understand how components reset to avoid unexpected behavior.

BacktestEngine.reset()

The `.reset()` method returns all stateful fields to their **initial value**, except for data and instruments which persist.

What gets reset:

- All trading state (orders, positions, account balances).
- Strategy instances are removed (you must re-add strategies before the next run).
- Engine counters and timestamps.

What persists:

- Data added via `.add_data()` (use `.clear_data()` to remove).
- Instruments (must match the persisted data).
- Venue configurations.

Instrument handling:

For `BacktestEngine`, instruments persist across resets by default (because data persists and instruments must match data). This is configured via

`CacheConfig.drop_instruments_on_reset=False` in the default `BacktestEngineConfig`.

Approaches for multiple backtest runs

There are two main approaches for running multiple backtests:

1

Use BacktestNode (recommended for production)

The high-level API is designed for multiple backtest runs with different configurations:

```
from nautilus_trader.backtest.node import BacktestNode
from nautilus_trader.config import BacktestRunConfig

# Define multiple run configurations
configs = [
    BacktestRunConfig(...), # Run 1
    BacktestRunConfig(...), # Run 2
    BacktestRunConfig(...), # Run 3
]

# Execute all runs
node = BacktestNode(configs=configs)
```

```
results = node.run()
```

Each run gets a fresh engine with clean state - no `reset()` needed.

2 Use `BacktestEngine.reset()`

For fine-grained control with the low-level API:

```
from nautilus_trader.backtest.engine import BacktestEngine

engine = BacktestEngine()

# Setup once
engine.add_venue(...)
engine.add_instrument(ETHUSDT)
engine.add_data(data)

# Run 1
engine.add_strategy(strategy1)
engine.run()

# Reset and run 2 - instruments and data persist
engine.reset()
engine.add_strategy(strategy2)
engine.run()

# Reset and run 3
engine.reset()
engine.add_strategy(strategy3)
engine.run()
```

i Instruments and data persist across resets by default for `BacktestEngine`, making parameter optimizations straightforward.

💡 Best practices

- For production backtesting: Use `BacktestNode` with configuration objects.
- For parameter optimizations: Use `BacktestEngine.reset()` to run multiple strategies against the same data.
- For quick experiments: Either approach works - choose based on individual use case.

Data

Data provided for backtesting drives the execution flow. Since a variety of data types can be used, it's crucial that your venue configurations align with the data being provided for

backtesting. Mismatches between data and configuration can lead to unexpected behavior during execution.

NautilusTrader is primarily designed and optimized for order book data, which provides a complete representation of every price level or order in the market, reflecting the real-time behavior of a trading venue. This provides the greatest execution granularity and realism. However, if granular order book data is either not available or necessary, then the platform has the capability of processing market data in the following descending order of detail:



1. Order Book Data/Deltas (L3 market-by-order):

- Full market depth with visibility of all individual orders.

2. Order Book Data/Deltas (L2 market-by-price):

- Market depth visibility across all price levels.

3. Quote Ticks (L1 market-by-price):

- Top of book only - best bid and ask prices and sizes.

4. Trade Ticks:

- Actual executed trades.

5. Bars:

- Aggregated trading activity over fixed time intervals (e.g., 1-minute, 1-hour, 1-day).

Choosing data: cost vs. accuracy

For many trading strategies, bar data (e.g., 1-minute) can be sufficient for backtesting and strategy development. This is particularly important because bar data is typically much more accessible and cost-effective compared to tick or order book data.

Given this practical reality, Nautilus is designed to support bar-based backtesting with advanced features that maximize simulation accuracy, even when working with lower granularity data.



For some trading strategies, it can be practical to start development with bar data to validate core trading ideas. If the strategy looks promising, but is more sensitive to precise execution timing (e.g., requires fills at specific prices between OHLC levels, or uses tight take-profit/stop-loss levels), you can then invest in higher granularity data for more accurate validation.

Venues

When initializing a venue for backtesting, you must specify its internal order `book_type` for execution processing from the following options:

- `L1_MBP`: Level 1 market-by-price (default). Only the top level of the order book is maintained.
- `L2_MBP`: Level 2 market-by-price. Order book depth is maintained, with a single order aggregated per price level.
- `L3_MBO`: Level 3 market-by-order. Order book depth is maintained, with all individual orders tracked as provided by the data.

The `book_type` determines which data types the matching engine uses to update book state and drive execution. Data types not applicable for a given `book_type` are ignored for book and price updates, though precision validation still applies and the engine clock still advances. Strategies always receive all subscribed data via the data engine regardless of `book_type`.

Data Type	L1_MBP	L2_MBP	L3_MBO
<code>QuoteTick</code>	Updates book	<i>Ignored</i>	<i>Ignored</i>
<code>TradeTick</code>	Triggers matching	Triggers matching	Triggers matching
<code>Bar</code>	Updates book	<i>Ignored</i>	<i>Ignored</i>
<code>OrderBookDelta</code>	<i>Ignored</i>	Updates book	Updates book
<code>OrderBookDeltas</code>	<i>Ignored</i>	Updates book	Updates book
<code>OrderBookDepth10</code>	Updates book	Updates book	Updates book



The granularity of the data must match the specified order `book_type`. Nautilus cannot generate higher granularity data (L2 or L3) from lower-level data such as quotes, trades, or bars.

⚠ If you specify `L2_MBP` or `L3_MBO` as the venue's `book_type`, quotes and bars will not update the book. Ensure you provide order book delta data, otherwise orders may appear as though they are never filled.

⚠ When using `L1_MBP` (the default), order book deltas are ignored by the matching engine. If you subscribe to order book deltas, set the venue `book_type` to `L2_MBP` or `L3_MBO`. This also applies to sandbox execution, where the matching engine uses the same `book_type` configuration.

Execution

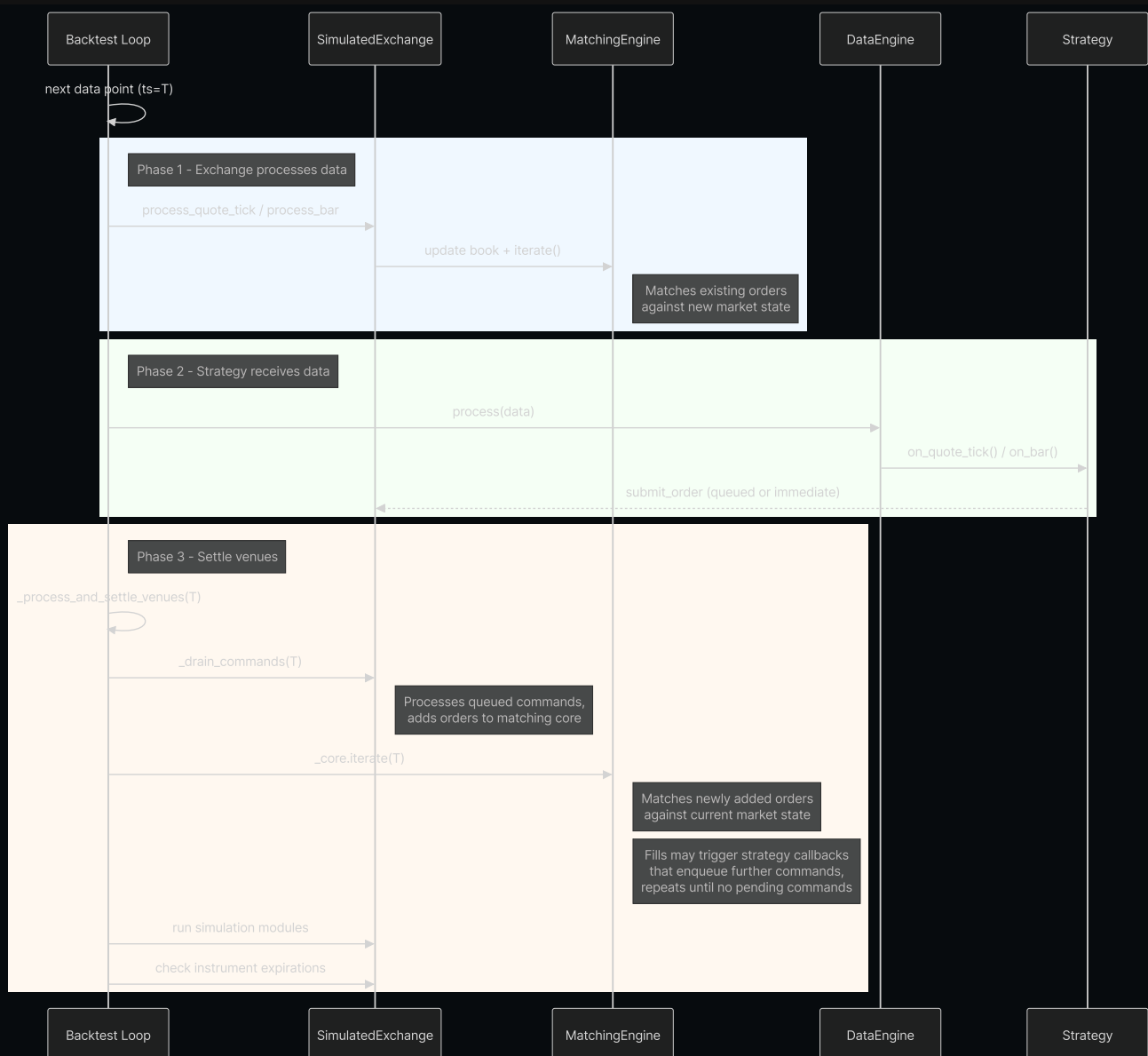
Data and message sequencing

In the main backtesting loop, new market data is processed for order execution before being dispatched to actors/strategies via the data engine.

Main loop flow

For each data point the engine runs three phases:

- **Exchange processes data.** The simulated exchange updates its order book from the incoming market data and iterates the matching engine. This fills any existing orders that now match against the new market state.
- **Strategy receives data.** The data engine dispatches the data point to actors and strategies via their callbacks (e.g. `on_quote_tick`, `on_bar`). Strategies may submit, cancel, or modify orders during these callbacks.
- **Settle venues.** The engine drains all queued venue commands and then iterates matching engines to fill newly submitted orders. This loop repeats until no pending commands remain, so cascading orders (e.g. a hedge submitted from `on_order_filled`) settle within the same timestamp.



Timer events use the same settle mechanism but batch by timestamp: all callbacks at timestamp T execute first, then venues are settled for T before advancing to $T+1$.

Command settling

When an order fill triggers a strategy callback that submits additional orders (e.g., a stop-loss submitted in `on_order_filled`), those cascading commands are settled within the same timestamp/event cycle. The engine repeatedly drains venue command queues and any newly generated commands until no commands remain pending for the current timestamp. Simulation modules are run only once per cycle, after all commands have settled.

When a `LatencyModel1` is configured, commands are placed in the venue's inflight queue with a future timestamp derived from the simulated latency. The settle loop considers inflight commands that are due at the current timestamp as pending, so zero-latency or same-tick latency configurations still settle correctly. Commands with future timestamps are deferred and processed when the engine reaches that time.

Shutdown semantics

`BacktestEngine::end()` invokes each strategy's `on_stop` handler, drains and settles any commands it emits (e.g. `close_all_positions`, `cancel_all_orders`), then stops the engines.

- Final positions, orders, and account balances reflect `on_stop` fills and cancels.
- Strategy event handlers do not fire for the resulting events: the strategy is already `Stopped`, so `OrderFilled` and similar events log but bypass `on_order_filled` and friends. Logic that reacts to fills must run before `on_stop` returns.
- Simulation modules do not re-run at shutdown. `SimulationModule::process` is once per timestamp; re-invoking would double-apply side effects like FX rollover interest.
- A `LatencyModel` defers `on_stop` commands by its configured delay; the engine clock does not advance past the final data point, so those commands stay pending. Close earlier (e.g. in the last `on_bar`) for reliable closing fills with latency.

Fill modeling philosophy

NautilusTrader treats historical order book and trade data as **immutable** during backtesting. What happened in the market is preserved exactly as recorded. Fills never modify the underlying book state.

This addresses a gap in academic literature: most research focuses on live market dynamics where the book actually evolves. Historical backtesting with frozen snapshots is a distinct engineering problem: how do we simulate realistic fills against data that doesn't change in response to our orders?

Design choices:

- **Immutable historical data:** Order book and trade data are never modified.
- **Optional consumption tracking:** When `liquidity_consumption=True`, the engine tracks consumed liquidity per price level to prevent duplicate fills. See [Order book immutability](#) for configuration.
- **Reproducible results:** A fixed `random_seed` pins the probabilistic fill model's PRNG. Same-process reruns are expected to match; cross-process reruns may differ in rare cases due to hash-ordering effects outside the fill model.

Fill price determination

The matching engine determines fill prices based on order type, book type, and market state.

L2/L3 order book data

With full order book depth, fills are determined by actual book simulation:

Order Type	Fill Price
MARKET	Walks the book, filling at each price level (taker).
MARKET_TO_LIMIT	Walks the book, filling at each price level (taker).
LIMIT	Order's limit price when matched (maker).
STOP_MARKET	Walks the book when triggered.
STOP_LIMIT	Order's limit price when triggered and matched.
MARKET_IF_TOUCHED	Walks the book when triggered.
LIMIT_IF_TOUCHED	Order's limit price when triggered.
TRAILING_STOP_MARKET	Walks the book when activated and triggered.
TRAILING_STOP_LIMIT	Order's limit price when activated, triggered, and matched.

With L2/L3 data, market-type orders may partially fill across multiple price levels if insufficient liquidity exists at the top of book. Limit-type orders act as resting orders after triggering and may remain unfilled if the market doesn't reach the limit price. **MARKET_TO_LIMIT** fills as a taker first, then rests any remaining quantity as a limit order at its first fill price.

L1 order book data (quotes, trades, bars)

With only top-of-book data, the same book simulation is used with a single-level book:

Order Type	BUY Fill Price	SELL Fill Price
MARKET	Best ask	Best bid
MARKET_TO_LIMIT	Best ask	Best bid
LIMIT	Limit price	Limit price

Order Type	BUY Fill Price	SELL Fill Price
STOP_MARKET	Best ask	Best bid
STOP_LIMIT	Limit price	Limit price
MARKET_IF_TOUCHED	Best ask	Best bid
LIMIT_IF_TOUCHED	Limit price	Limit price
TRAILING_STOP_MARKET	Best ask	Best bid
TRAILING_STOP_LIMIT	Limit price	Limit price

With L1 data, the simulated book has a single price level. Orders fill against the available size at that level. If an order has remaining quantity after exhausting top-of-book liquidity, market and marketable limit-style orders will slip one tick to fill the residual.

For bar data specifically, `STOP_MARKET` and `TRAILING_STOP_MARKET` orders may fill at the trigger price rather than best ask/bid when the bar moves through the trigger during its high/low processing. See [Stop order fill behavior with bar data](#) for details.

i Fill models can alter these fill prices. See the [Fill models](#) section for details on configuring execution simulation.

Order type semantics

- **Market execution:** Fill at current market price (bid/ask). This models real exchange behavior where these orders execute at the best available price after triggering. Exception: with bar data, `STOP_MARKET` and `TRAILING_STOP_MARKET` orders triggered during H/L processing fill at the trigger price (see below).
- **Limit execution:** Fill at the order's limit price when matched. Provides price guarantee but may not fill if the market doesn't reach the limit.

Stop order fill behavior with bar data

When backtesting with bar data only (no tick data), the matching engine distinguishes between two scenarios for `STOP_MARKET` and `TRAILING_STOP_MARKET` orders:

Gap scenario (bar opens past trigger): When a bar's open price gaps past the trigger price, the stop triggers immediately and fills at the market price (the open). This models real

exchange behavior where stop-market orders provide no price guarantee during gaps.

Example - SELL `STOP_MARKET` with trigger at 100:

- Previous bar closes at 105.
- Next bar opens at 90 (overnight gap down).
- Stop triggers at open and fills at 90.

Move-through scenario (bar moves through trigger): When a bar opens normally and then its high or low moves through the trigger price, the stop fills at the trigger price. Since we only have OHLC data, we assume the market moved smoothly through the trigger and the order would have filled there.

Example - SELL `STOP_MARKET` with trigger at 100:

- Bar opens at 102 (no gap).
- Bar low reaches 98, moving through trigger at 100.
- Stop fills at 100 (the trigger price).

This behavior caps potential slippage during orderly market moves while still modeling gap slippage accurately. For tick-level precision, use quote or trade tick data instead of bars.

Price protection

Price protection defines an exchange-calculated price boundary that prevents marketable orders from executing at excessively aggressive prices. This models exchanges like Binance and CME that implement protection mechanisms for market and stop-market orders.

Configuration:

```
from nautilus_trader.backtest.config import BacktestVenueConfig

venue_config = BacktestVenueConfig(
    name="BINANCE",
    oms_type="NETTING",
    account_type="MARGIN",
    starting_balances=["100_000 USDT"],
    price_protection_points=100, # 100 points = 1.00 offset for 2-decimal instruments
)
```



How it works:

The matching engine calculates the protection boundary from the current best bid/ask at fill time:

- **BUY orders:** $\text{protection_price} = \text{ask} + (\text{points} \times \text{price_increment})$
- **SELL orders:** $\text{protection_price} = \text{bid} - (\text{points} \times \text{price_increment})$

The engine filters out fills beyond the protection boundary. For example, with `price_protection_points=100` on an instrument with `price_increment=0.01`:

- Best ask is 1001.00.
- Protection price = $1001.00 + (100 \times 0.01) = 1002.00$.
- A BUY market order fills only at prices ≤ 1002.00 .
- Liquidity at 1003.00 or higher is filtered, leaving the order partially filled.

Trigger-time semantics:

The engine computes protection at fill time, not order submission time:

- **Market orders:** Protection computed immediately when the order processes.
- **Stop-market orders:** Protection computed when the stop triggers, using the bid/ask at that moment.

This design allows stop orders to be submitted even when the opposite side of the book is empty, since the engine computes protection later when the stop triggers.

Order types affected:

- `MARKET`
- `STOP_MARKET`

Limit orders are unaffected since they already define a price boundary.

 Set `price_protection_points=0` to disable price protection (default behavior).

Slippage and spread handling

When backtesting with different types of data, Nautilus implements specific handling for slippage and spread simulation:

For L2 (market-by-price) or L3 (market-by-order) data, slippage is simulated with high accuracy by:

- Filling orders against actual order book levels.
- Matching available size at each price level sequentially.
- Maintaining realistic order book depth impact (per order fill).

For L1 data types (e.g., L1 order book, trades, quotes, bars), slippage is handled through the `FillModel`:

Per-fill slippage (`prob_slippage`):

- Applies to each fill when using an L1 book with a configured `FillModel`.
- Affects all order types (market, limit, stop, etc.).
- When triggered, moves the fill price one tick against the order direction.
- Example: With `prob_slippage=0.5`, a BUY order has 50% chance of filling one tick above the best ask.

i When backtesting with bar data, be aware that the reduced granularity of price information affects the slippage mechanism. For the most realistic backtesting results, consider using higher granularity data sources such as L2 or L3 order book data when available.

How simulation varies by data type

The behavior of the `FillModel` adapts based on the order book type being used:

L2/L3 order book data

With full order book depth, the `FillModel` focuses purely on simulating queue position for limit orders through `prob_fill_on_limit`. The order book itself handles slippage naturally based on available liquidity at each price level.

- `prob_fill_on_limit` is active - simulates queue position.
- `prob_slippage` is not used - real order book depth determines price impact.

⚠ The historical order book is immutable during backtesting. Book depth is **not** decremented after fills. By default (`liquidity_consumption=False`), the same liquidity can be consumed repeatedly within an iteration. Enable `liquidity_consumption=True` to track consumed liquidity per price level. Consumption resets when fresh data arrives at that level. See [Order book immutability](#) for details.

L1 order book data

With only best bid/ask prices available, the `FillModel` provides additional simulation:

- `prob_fill_on_limit` is active - simulates queue position.
- `prob_slippage` is active - simulates basic price impact since we lack real depth information.

Bar/Quote/Trade data

When using less granular data, the same behaviors apply as L1:

- `prob_fill_on_limit` is active - simulates queue position.
- `prob_slippage` is active - simulates basic price impact.

Important considerations

- **Partial fills:** With L2/L3 data, fills are limited to available liquidity at each price level. With L1 data, the full order quantity fills at the single available level.
- **Consumption tracking:** See [Order book immutability](#) for details on preventing duplicate fills.

Order book immutability

Historical order book data is immutable during backtesting. When your order fills against book liquidity, the book state remains unchanged. This preserves historical data integrity.

The matching engine can optionally use **per-level consumption tracking** to prevent duplicate fills while allowing fills when fresh liquidity arrives. This behavior is controlled by the `liquidity_consumption` configuration option.

Configuration:

```
from nautilus_trader.backtest.config import BacktestVenueConfig

venue_config = BacktestVenueConfig(
    name="SIM",
    oms_type="NETTING",
```



```

account_type="CASH",
starting_balances=["100_000 USD"],
liquidity_consumption=True, # Enable consumption tracking (default: False)
)

```

- `liquidity_consumption=False` (default): Each iteration fills against the full book liquidity independently. Simpler behavior, assumes you're a small participant whose orders don't meaningfully impact available liquidity.
- `liquidity_consumption=True`: Tracks consumed liquidity per price level. Prevents the same displayed liquidity from generating multiple fills. Resets when fresh data arrives at that level.

How consumption tracking works (when enabled):

For each price level, the engine maintains:

- `original_size`: The book's quantity when tracking began.
- `consumed`: How much has been filled against this level.

When processing a fill:

1. Check if the book's current size at this level matches `original_size`
2. If different (fresh data arrived), reset the entry: `original_size = current_size`, `consumed = 0`
3. Calculate `available = original_size - consumed`
4. After filling, increment `consumed` by the fill quantity

Example:

1. Order book shows 100 units at ask 100.00. Engine tracks: `(original=100, consumed=0)`.
2. Your BUY order fills 30 units. Engine updates: `(original=100, consumed=30)`. Available = 70.
3. Another BUY order attempts 50 units. Available = 70, so it fills 50. `(original=100, consumed=80)`.
4. A delta updates ask 100.00 to 120 units. Engine resets: `(original=120, consumed=0)`.
5. New orders can now fill against the fresh 120 units.

Passive limit order fills on L1 data:

With L1 data (quotes, trades, bars), the book has only a single price level per side. When the market moves through a passive (MAKER) limit order's price, the engine must decide how to handle remaining order quantity after exhausting displayed liquidity.

<code>liquidity_consumption</code>	Behavior when market moves through passive limit
<code>False</code> (default)	Fill entire order at limit price. Assumes market movement implies sufficient liquidity existed.
<code>True</code>	Fill only against displayed liquidity. Order remains open for subsequent fills.

Example scenario (`liquidity_consumption=True`):

1. Quote shows ask 100.10 with 50 units.
2. You place BUY LIMIT at 100.05 for 1000 units (passive, resting below ask).
3. Next quote shows ask 100.00 with 30 units (market moved through your limit).
4. Order fills 30 units against displayed liquidity. 970 units remain open.
5. Next quote shows ask 99.95 with 200 units.
6. Order fills another 200 units. 770 units remain open.
7. Fills continue as fresh liquidity arrives at crossed price levels.

This behavior provides conservative fill simulation: your order only fills against liquidity actually observed in the data, rather than inferring liquidity from price movements.

Trade tick liquidity:

Trade ticks provide evidence of executable liquidity at the trade price. When a trade occurs at a price level not reflected in the current book, the engine can use the trade quantity as available liquidity, subject to the same consumption tracking rules (when enabled).

Trade consumption seeding:

When using L2/L3 book data and a trade tick triggers order matching (e.g., triggering a resting stop order), the trade itself consumed liquidity from the book. Before simulating fills for triggered orders, the engine pre-seeds the consumption maps with the trade's consumed volume. This prevents triggered orders from filling against liquidity that the triggering trade

already consumed. This seeding is skipped for L1 books, where the trade tick has already updated the single top-of-book level directly.

For example, if the book has 10 units at the best ask and a BUY trade of size 8 triggers a stop market BUY for 5 units, the stop order sees only 2 units remaining at best ask ($10 - 8$) and must fill the remaining 3 units at the next price level. Without this seeding, the stop would incorrectly fill all 5 units at the best ask price.

The engine uses a timestamp guard to avoid double-counting: if the book's most recent update (`ts_last`) is newer than the trade's event time (`ts_event`), seeding is skipped. This handles exchanges like Binance where depth deltas arrive before the corresponding trade tick, so the book already reflects the consumed liquidity, so additional seeding would over-penalize fills.

 As the `FillModel` continues to evolve, future versions may introduce more sophisticated simulation of order execution dynamics, including:

- Variable slippage based on order size.
- More complex queue position modeling.

Known limitations

No queue position within a level: Consumption tracking determines *how much* liquidity remains at a level, but doesn't model *where* your order sits in the queue relative to other participants. Use `prob_fill_on_limit` to simulate queue position probabilistically.

Trade-driven fills are opportunistic: When trade ticks indicate liquidity at a price not in the book, the engine uses this as fill evidence. However, this represents liquidity that existed momentarily and may not reflect sustained availability.

Trade-based execution

Trade tick data triggers order fills by default (`trade_execution=True`). A trade tick indicates that liquidity was accessed at the trade price, allowing resting limit orders to match. This mirrors the default behavior for bar data (`bar_execution=True`).

Advanced users who want to isolate execution to L1 book data only (quotes or order book updates) can disable trade-based execution:

```
venue_config = BacktestVenueConfig(
```



```
name="SIM",  
oms_type="NETTING",  
account_type="CASH",  
starting_balances=["100_000 USD"],  
trade_execution=False, # Disable trade-based fills  
)
```

When `trade_execution=False` or `bar_execution=False`, the respective data types skip order matching and maintenance operations (GTD order expiry, trailing stop activation, instrument expiration checks). Quote ticks always trigger maintenance, so this is typically acceptable when using multiple data types.

The matching engine uses a "transient override" mechanism: during the matching process, it temporarily adjusts the matching core's Best Bid (for BUYER trades) or Best Ask (for SELLER trades) toward the trade price. This allows resting orders on the passive side to cross the spread and fill. Note: the underlying order book data is never modified (it remains immutable); only the matching core's internal price references are adjusted.

Fill determination:

When a trade tick triggers order matching, the engine determines fills as follows:

1. **Book reflects trade price:** If the order book has liquidity at the trade price, fills use book depth (standard behavior).
2. **Book doesn't reflect trade price:** If the book's liquidity is at a different price, the engine uses a "trade-driven fill" at the order's limit price, capped to `min(order.leaves_qty, trade.size)`.

This ensures that when a trade prints through the spread but the book hasn't updated, fills are bounded by what the trade tick actually evidences. When `liquidity_consumption=False` (default), the same trade size can fill multiple orders within an iteration. When `liquidity_consumption=True`, consumption tracking applies to trade-driven fills as well. Repeated fills at the same trade price will be bounded by consumed liquidity until fresh data arrives.

Restoration behavior:

After matching, the core's bid/ask are only restored to their original values if the trade price improved them (moved them away from the spread):

- **SELLER trade:** Ask is restored only if trade price was below the original ask.

- **BUYER trade:** Bid is restored only if trade price was above the original bid.

If the trade price didn't improve the quote (e.g., a SELLER trade at or above the ask), the core retains the trade price. This means repeated trades at or beyond the spread can progressively move the core's bid/ask.

Fill price:

- **SELLER trade at P:** The engine sets the core's Best Ask to P (if $P < \text{current ask}$). Resting BUY LIMIT orders at P or higher will fill at their limit price (if book doesn't have that level) or at book prices (if book does).
- **BUYER trade at P:** The engine sets the core's Best Bid to P (if $P > \text{current bid}$). Resting SELL LIMIT orders at P or lower will fill at their limit price (if book doesn't have that level) or at book prices (if book does).

This conservative approach ensures fills occur at the order's limit price rather than potentially better trade prices. For example, a BUY LIMIT at 100.05 triggered by a SELLER trade at 100.00 will fill at 100.05, not 100.00.



Combine trade data with book or quote data for best results: book/quote data establishes the baseline spread, while trade ticks trigger execution for orders that might be inside the spread or ahead of the quote updates.

Understanding trade tick aggressor sides

A common source of confusion is the `aggressor_side` field on trade ticks:

- **SELLER trade:** A seller aggressed, selling into the bid. This provides evidence of fill-able liquidity for **BUY** orders at the trade price.
- **BUYER trade:** A buyer aggressed, buying from the ask. This provides evidence of fill-able liquidity for **SELL** orders at the trade price.

In other words, trade ticks trigger fills for orders on the **opposite** side of the aggressor. A SELLER trade at 100.00 can fill your resting BUY LIMIT at 100.00, but cannot fill your SELL LIMIT, since the trade already represents someone else selling.

Combining L2 book data with trade ticks

When using L2 order book data (e.g., 100ms throttled depth snapshots) combined with trade tick data:

1. **Book updates establish the spread:** Each book delta/snapshot updates the matching engine's view of available liquidity at each price level.
2. **Trade ticks provide execution evidence:** Trade ticks indicate that liquidity was accessed at a specific price, potentially between book snapshots.
3. **Fill quantity determination:** When a trade triggers a fill:
 - If the book already reflects liquidity at the trade price, fills use book depth
 - If the trade price is inside the spread (not in the current book), fills are capped by `min(order.leaves_qty, trade.size)`
4. **Timing considerations:** With throttled book data (e.g., 100ms), the book may lag behind trades. A trade at a price not yet reflected in the book will use trade-driven fill logic.

Common misconception: Users sometimes expect every trade tick to trigger fills. Remember:

- Only trades on the **opposite** side can fill your orders.
- SELLER trades -> potential BUY fills.
- BUYER trades -> potential SELL fills.
- Book UPDATE events move the market but only trigger fills if prices cross your order.

Queue position tracking

When `queue_position=True` is enabled alongside `trade_execution=True`, the matching engine simulates queue position for limit orders. This provides more realistic fill behavior by tracking how many orders are "ahead" of your order at a given price level.

How it works:

1. **Order placement:** When a LIMIT order is accepted, the engine snapshots the current same-side book depth at the order's price level. This represents the orders ahead in the queue.
2. **Trade ticks:** When trade ticks occur at the order's price level, the "quantity ahead" is decremented by the trade size. Only trades on the correct side affect the queue (BUYER trades decrement queue for SELL orders, SELLER trades decrement queue for BUY orders). Trades with `NO_AGGRESSOR` (common in historical datasets lacking aggressor

metadata) affect both sides. This is pessimistic but prevents orders from stalling indefinitely.

3. **Fill eligibility:** The order becomes eligible to fill only when the quantity ahead reaches zero. On the tick that clears the queue, only the excess volume (trade size minus queue ahead) is available for fill, preventing overfill.
4. **Price level DELETE:** If the order book level is deleted (BookAction.DELETE), the queue clears immediately, making the order fill-eligible. UPDATE actions are ignored (queue unchanged).
5. **Order modification:** If the order is modified (price or quantity change), the queue position resets. The order moves to the back of the queue at its new price level.

Configuration:

```
from nautilus_trader.backtest.config import BacktestVenueConfig

venue_config = BacktestVenueConfig(
    name="SIM",
    oms_type="NETTING",
    account_type="MARGIN",
    starting_balances=["100_000 USD"],
    trade_execution=True,          # Required for queue_position
    queue_position=True,          # Enable queue position tracking
)
```

Example scenario:

1. Order book shows 100 units at bid 100.00.
2. You place a BUY LIMIT at 100.00 for 50 units. Queue ahead = 100.
3. SELLER trade of 80 units at 100.00 -> queue ahead = 20. No fill yet.
4. SELLER trade of 30 units at 100.00 -> queue clears with 10 excess. Fill = 10 units.
5. Next SELLER trade of 50 units -> fill remaining 40 units.

Limitations:

- Only applies to **LIMIT** orders. Stop-limit and limit-if-touched orders are not tracked in this implementation.
- Queue position is per-order, not shared across multiple orders at the same price.

- The queue snapshot is based on book state at order acceptance time.
- Trades with `NO_AGGRESSOR` decrement queue for both sides, which may cause orders to fill sooner than in reality (pessimistic for queue estimation, but prevents stalling).

L1 quote-based mode:

When using `BookType.L1_MBP` (top-of-book quotes only), queue position tracking uses trade ticks to decrement the queue (the same mechanism as L2/L3), while quote ticks handle price-move detection and deferred snapshot resolution.

- **Trade ticks:** Trades at the order's price level decrement the queue ahead by the trade size, identical to L2/L3 behavior. Only trades on the correct aggressor side affect the queue (SELLER trades decrement queue for BUY orders, BUYER trades for SELL orders).
- **Price moves away:** If the bid drops below a BUY order's price (or ask rises above a SELL order's price), the order's price level has been "crossed" and the queue clears to zero, making the order fill-eligible on the next matching trade.
- **Price moves toward:** If the bid rises (or ask drops), the level at the order's price was not consumed, so queue positions are preserved.
- **Price returns to a level:** When the price returns after moving away, the queue ahead is capped at the new displayed size if it was previously larger.
- **Orders behind BBO (pending):** When a limit order is placed behind the best bid/ask (e.g., BUY below best bid), the queue snapshot is deferred because L1 data has no visible depth at that level. Fills are blocked until the BBO reaches the order's price, at which point the queue is snapshotted from the displayed size. Pending orders are also resolved when trades cross through their price level.

L1 mode uses the same configuration: set `queue_position=True` with `book_type=BookType.L1_MBP`. This provides a lightweight alternative to full L2/L3 data when only top-of-book quotes are available.

i Queue position tracking provides a heuristic simulation of queue dynamics. Real exchange queue behavior depends on many factors (order priority rules, hidden orders, etc.) that cannot be perfectly reconstructed from historical data.

Bar-based execution

Bar data provides a summary of market activity with four key prices for each time period (assuming bars are aggregated by trades):

- **Open:** opening price (first trade)
- **High:** highest price traded
- **Low:** lowest price traded
- **Close:** closing price (last trade)

While this gives us an overview of price movement, we lose some important information that we'd have with more granular data:

- We don't know in what order the market hit the high and low prices.
- We can't see exactly when prices changed within the time period.
- We don't know the actual sequence of trades that occurred.

This is why Nautilus processes bar data through a system that attempts to maintain the most realistic yet conservative market behavior possible, despite these limitations. At its core, the platform always maintains an order book simulation - even when you provide less granular data such as quotes, trades, or bars (although the simulation will only have a top level book).

 When using bars for execution simulation (enabled by default with `bar_execution=True` in venue configurations), Nautilus strictly expects the initialization timestamp (`ts_init`) of each bar to represent its **closing time**. This ensures accurate chronological processing, prevents look-ahead bias, and aligns market updates (Open -> High -> Low -> Close) with the moment the bar is complete.

The event timestamp (`ts_event`) can represent either the open or close time of the bar:

- If `ts_event` is at the **close**, ensure `ts_init_delta=0` when processing bars (default).
- If `ts_event` is at the **open**, set `ts_init_delta` equal to the bar's duration to shift `ts_init` to the close.

Bar timestamp convention

If your data source provides bars timestamped at the **opening time** (common in some providers), you need to ensure `ts_init` is set to the closing time for correct execution simulation. There are two approaches:

Approach 1: Adjust data timestamps (recommended)

- Use adapter-specific configurations like `bars_timestamp_on_close=True` (e.g., for Bybit or Databento adapters) to handle this automatically during data ingestion.
- For custom data, manually shift the timestamps by the bar duration before loading (e.g., add 1 minute for `1-MINUTE` bars).
- This approach is clearest because the data itself reflects the close time.

Approach 2: Use `ts_init_delta` parameter

- When calling `BarDataWrangler.process()`, set `ts_init_delta` to the bar's duration in nanoseconds (e.g., `60_000_000_000` for 1-minute bars).
- The wrangler computes `ts_init = ts_event + ts_init_delta`, shifting execution timing to the close.
- Use this when you cannot or prefer not to modify source data timestamps.

Always verify your data's timestamp convention with a small sample to avoid simulation inaccuracies. Incorrect timestamp handling can lead to look-ahead bias and unrealistic backtest results.

Processing bar data

Even when you provide bar data, Nautilus maintains an internal order book for each instrument, as a real venue would.

1. Time processing:

- Nautilus has a specific way of handling the timing of bar data *for execution* that's crucial for accurate simulation.
- The initialization timestamp (`ts_init`) is used for execution timing and must represent the close time of the bar. This approach is most logical because it represents the moment when the bar is fully formed and its aggregation is complete.
- The event timestamp (`ts_event`) represents when the data event occurred and may differ from `ts_init` depending on your data source:
 - If your bars are timestamped at the **close** (the recommended default), use `ts_init_delta=0` in `BarDataWrangler` so that `ts_init = ts_event`.
 - If your bars are timestamped at the **open**, set `ts_init_delta` to the bar's duration in nanoseconds (e.g., `60_000_000_000` for 1-minute bars) to shift `ts_init` to the close

time.

- The platform ensures all events happen in the correct sequence based on `ts_init`, preventing any possibility of look-ahead bias in your backtests.

i Exceptions for bar execution

Bars will **not** be processed for execution (and will not update the order book) in the following cases:

- **Internally aggregated bars:** Bars with `AggregationSource.INTERNAL` are skipped to avoid processing bars that are derived from already-processed tick data.
- **Non-L1 book types:** When the venue's `book_type` is configured as `L2_MBP` or `L3_MBO`, bar data is ignored for execution processing, as bars are derived from top-of-book prices only.

In these cases, bars will still be received by strategies for analytics and decision-making, but they won't trigger order matching or update the simulated order book.

2. Price processing:

- The platform converts each bar's OHLC prices into a sequence of market updates.
- By default, updates follow the order: Open -> High -> Low -> Close (configurable via `bar_adaptive_high_low_ordering`).
- If you provide multiple timeframes (like both 1-minute and 5-minute bars), the platform uses the more granular data for highest accuracy.

3. Executions:

- When you place orders, they interact with the simulated order book as they would on a real venue.
- For MARKET orders, execution happens at the current simulated market price plus any configured latency.
- For LIMIT orders working in the market, they'll execute if any of the bar's prices reach or cross your limit price (see below).
- The matching engine continuously processes orders as OHLC prices move, rather than waiting for complete bars.

OHLC prices simulation

During backtest execution, each bar is converted into a sequence of four price points:

1. Opening price

2. High price (*Order between High/Low is configurable. See `bar_adaptive_high_low_ordering` below.*)
3. Low price
4. Closing price

The trading volume for that bar is **split evenly** among these four points (25% each), with any remainder added to the closing price trade to preserve total volume. In marginal cases, if the bar's volume divided by 4 is less than the instrument's minimum `size_increment`, we use the minimum `size_increment` per price point to ensure valid market activity (e.g., 1 contract for CME group exchanges).

How these price points are sequenced can be controlled via the `bar_adaptive_high_low_ordering` parameter when configuring a venue.

Nautilus supports two modes of bar processing:

1. Fixed ordering (`bar_adaptive_high_low_ordering=False`, default)

- Processes every bar in a fixed sequence: `Open -> High -> Low -> Close`.
- Simple and deterministic approach.

2. Adaptive ordering (`bar_adaptive_high_low_ordering=True`)

- Uses bar structure to estimate likely price path:
 - If Open is closer to High: processes as `Open -> High -> Low -> Close`.
 - If Open is closer to Low: processes as `Open -> Low -> High -> Close`.
- [Research](#) shows this approach achieves ~75-85% accuracy in predicting correct High/Low sequence (compared to statistical ~50% accuracy with fixed ordering).
- This is particularly important when both take-profit and stop-loss levels occur within the same bar - as the sequence determines which order fills first.

Here's how to configure adaptive bar ordering for a venue, including account setup:

```
from nautilus_trader.backtest.engine import BacktestEngine
from nautilus_trader.model.enums import OmsType, AccountType
from nautilus_trader.model import Money, Currency

# Initialize the backtest engine
```



```
engine = BacktestEngine()

# Add a venue with adaptive bar ordering and required account settings
engine.add_venue(
    venue=venue, # Your Venue identifier, e.g., Venue("BINANCE")
    oms_type=OmsType.NETTING,
    account_type=AccountType.CASH,
    starting_balances=[Money(10_000, Currency.from_str("USDT"))],
    bar_adaptive_high_low_ordering=True, # Enable adaptive ordering of High/Low bar prices
)
```

Order submission timing

Bar N's OHLC sequence processes before `on_bar(N)` fires. Without a `LatencyModel`, an order submitted from `on_bar` settles immediately and matches against the current book, whose top reflects bar N's close.

Attach a `LatencyModel` to the venue to defer the order's effective arrival. With bar-only data and no intervening timer events, the order settles after the next bar's OHLC sweep, so the fill price is that bar's close (or a later bar's close if latency exceeds the bar interval). Finer-grained data (quotes, trades) or timer-driven settlement between bars can drain the order earlier, against the book as it stands at that point:

```
from nautilus_trader.backtest.models import LatencyModel

engine.add_venue(
    venue=venue,
    oms_type=OmsType.NETTING,
    account_type=AccountType.CASH,
    starting_balances=[Money(10_000, Currency.from_str("USDT"))],
    latency_model=LatencyModel(base_latency_nanos=1_000_000_000), # 1 second
)
```

i A native "next-bar-open" execution mode is not provided. A bar's `ts_init` is its close timestamp, so the open price is only known once the bar arrives. Filling at that open from a signal generated on the prior bar would require look-ahead.

Internal bar aggregation timing

When aggregating time bars internally from tick data, the data engine uses timers to close bars at interval boundaries. A timing edge case occurs when data arrives at the exact bar close timestamp: the timer may fire before processing boundary data.

Configure `timeBarsBuildDelay` in `DataEngineConfig` to delay bar close timers:


```
from nautilus_trader.config import BacktestEngineConfig
from nautilus_trader.data.config import DataEngineConfig

config = BacktestEngineConfig(
    data_engine=DataEngineConfig(
        timeBarsBuildDelay=1, # Microseconds
    ),
)
```



A small delay (1 microsecond) ensures boundary data is processed before the bar closes. Useful when tick data clusters at round interval timestamps.



Only affects internally aggregated bars (`AggregationSource.INTERNAL`).

Timer-only backtests

The backtest engine supports running with timers but no market data. This is useful for scheduled operations or testing timer-based logic. Timers fire in chronological order, and timer callbacks can dynamically add data via `add_data_iterator()` which will be processed in sequence.



Data added by timer callbacks at the exact start time should have timestamps **after** the start time. The engine reads the first data point before processing start-time timers, so dynamically added data with timestamps at or before the start time may not be processed in the expected order.

Fill models

Fill models simulate order execution dynamics during backtesting. They address a fundamental challenge: *even with perfect historical market data, we can't fully simulate how orders may have interacted with other market participants in real-time.*

The base `FillModel` provides probabilistic parameters for queue position and slippage simulation. Subclasses can override `get_orderbook_for_fill_simulation()` to generate synthetic order books for more sophisticated liquidity modeling.

Available fill models

Model	Description	Use Case
<code>FillModel</code>	Base model with probabilistic fill/slippage parameters.	Simple queue position and slippage.
<code>BestPriceFillModel</code>	Fills at best price with unlimited liquidity.	Testing basic strategy logic optimistically.
<code>OneTickSlippageFillModel</code>	Forces exactly one tick of slippage on all orders.	Conservative slippage testing.
<code>TwoTierFillModel</code>	10 contracts at best price, remainder one tick worse.	Basic market depth simulation.
<code>ThreeTierFillModel</code>	50/30/20 contracts across three price levels.	More realistic depth simulation.
<code>ProbabilisticFillModel</code>	50% chance best price, 50% chance one tick slippage.	Randomized execution quality.
<code>SizeAwareFillModel</code>	Different execution based on order size (≤ 10 vs > 10).	Size-dependent market impact.
<code>LimitOrderPartialFillModel</code>	Max 5 contracts fill per price touch.	Queue position via partial fills.
<code>MarketHoursFillModel</code>	Wider spreads during low liquidity periods.	Session-aware execution.
<code>VolumeSensitiveFillModel</code>	Liquidity based on recent trading volume.	Volume-adaptive depth.
<code>CompetitionAwareFillModel</code>	Only percentage of visible liquidity available.	Multi-participant competition.

Configuring fill models

Using the base `FillModel` with probabilistic parameters:

```
from nautilus_trader.backtest.config import BacktestVenueConfig
from nautilus_trader.backtest.config import ImportableFillModelConfig

venue_config = BacktestVenueConfig(
    name="SIM",
    oms_type="NETTING",
    account_type="CASH",
```



```

starting_balances=["100_000 USD"],
fill_model=ImportableFillModelConfig(
    fill_model_path="nautilus_trader.backtest.models:FillModel",
    config_path="nautilus_trader.backtest.config:FillModelConfig",
    config={
        "prob_fill_on_limit": 0.2,      # Chance a limit order fills when price matches
        "prob_slippage": 0.5,          # Chance of 1-tick slippage (L1 data only)
        "random_seed": 42,             # Optional: Set for reproducible results
    },
),
)

```

Using an order book simulation model:

```

from nautilus_trader.backtest.config import BacktestVenueConfig
from nautilus_trader.backtest.config import ImportableFillModelConfig

venue_config = BacktestVenueConfig(
    name="SIM",
    oms_type="NETTING",
    account_type="CASH",
    starting_balances=["100_000 USD"],
    fill_model=ImportableFillModelConfig(
        fill_model_path="nautilus_trader.backtest.models:ThreeTierFillModel",
    ),
)

```

Probabilistic parameters (base FillModel)

prob_fill_on_limit (default: 1.0)

Simulates queue position by controlling the probability of a limit order filling when its price level is touched (but not crossed).

- 0.0: Never fills at touch (back of queue).
- 0.5: 50% chance of filling (middle of queue).
- 1.0: Always fills at touch (front of queue).

prob_slippage (default: 0.0)

Simulates price slippage on each fill. Only applies to L1 data types (quotes, trades, bars) where real depth is unavailable. Affects all order types when executing as takers.

- 0.0: No slippage (fills at best price).
- 0.5: 50% chance of one tick slippage per fill.

- `1.0`: Always slips one tick.

Order book simulation models

These models override the `get_orderbook_for_fill_simulation()` method to generate synthetic order books representing expected market liquidity. The matching engine fills orders against this simulated book.

How it works:

1. Before processing a fill, the matching engine calls `get_orderbook_for_fill_simulation()`.
2. If the model returns a synthetic order book, fills execute against that book's liquidity.
3. If the model returns `None`, standard fill logic applies.

i When a custom fill model provides a simulated order book, the `liquidity_consumption` tracking is **not** applied. Custom fill models are expected to manage their own liquidity simulation within the returned order book. Liquidity consumption tracking only affects the built-in fill logic (when `get_orderbook_for_fill_simulation()` returns `None`).

Example: ThreeTierFillModel

This model creates a book with liquidity distributed across three price levels:

- 50 contracts at best price
- 30 contracts one tick worse
- 20 contracts two ticks worse

A 100-contract market order would fill partially at each level, experiencing realistic price impact.

Creating custom fill models:

```
from nautilus_trader.backtest.models import FillModel
from nautilus_trader.model.book import OrderBook, BookOrder
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.core.rust.model import BookType

class MyCustomFillModel(FillModel):
    def get_orderbook_for_fill_simulation(
        self,
        instrument,
        order,
```



```

        best_bid,
        best_ask,
    ):
        book = OrderBook(
            instrument_id=instrument.id,
            book_type=BookType.L2_MBP,
        )

        # Add custom liquidity based on your market model
        # ...

    return book

```

Precision requirements and invariants

The matching engine enforces strict precision invariants to ensure data integrity throughout the fill pipeline. All prices and quantities must match the instrument's configured precision (`price_precision` and `size_precision`). Mismatches raise a `RuntimeError` immediately, preventing silent corruption of fill quantities.

Data/Operation	Field	Required Precision	Validation Location
QuoteTick	<code>bid_price</code> , <code>ask_price</code>	<code>instrument.price_precision</code>	<code>process_quote_tick</code>
QuoteTick	<code>bid_size</code> , <code>ask_size</code>	<code>instrument.size_precision</code>	<code>process_quote_tick</code>
TradeTick	<code>price</code>	<code>instrument.price_precision</code>	<code>process_trade_tick</code>
TradeTick	<code>size</code>	<code>instrument.size_precision</code>	<code>process_trade_tick</code>
Bar	<code>open</code> , <code>high</code> , <code>low</code> , <code>close</code>	<code>instrument.price_precision</code>	<code>process_bar</code>
Bar	<code>volume</code> (base units)	<code>instrument.size_precision</code>	<code>process_bar</code>
Order	<code>quantity</code>	<code>instrument.size_precision</code>	<code>process_order</code>
Order	<code>price</code>	<code>instrument.price_precision</code>	<code>process_order</code>
Order	<code>trigger_price</code>	<code>instrument.price_precision</code>	<code>process_order</code>
Order	<code>activation_price</code> *	<code>instrument.price_precision</code>	<code>process_order</code>
Order update	<code>quantity</code>	<code>instrument.size_precision</code>	<code>update_order</code>

Data/Operation	Field	Required Precision	Validation Location
Order update	<code>price</code> , <code>trigger_price</code>	<code>instrument.price_precision</code>	<code>update_order</code>
Fill	<code>fill_qty</code>	<code>instrument.size_precision</code>	<code>apply_fills</code> , <code>fill_order</code>
Fill	<code>fill_px</code>	<code>instrument.price_precision</code>	<code>apply_fills</code>

* `activation_price` is immutable after order submission.



`Bar.volume` must be in **base currency units**. Some data providers report quote-currency volume; convert to base units before loading (divide by price or use provider-specific fields).



If you encounter a precision mismatch error, align your data to the instrument:

```
# Align price/quantity to instrument precision
price = instrument.make_price(raw_price)
qty = instrument.make_qty(raw_qty)
```



Also verify that:

1. The instrument definition matches your data source's precision.
2. Data was not inadvertently rounded or truncated during loading.
3. Custom data loaders preserve the original precision metadata.

Accounts

Every backtest venue is attached with one of three `account_type` values: `CASH`, `MARGIN`, or `BETTING`. For the full data model, query API, and margin model reference, see [Accounting](#).

Example of adding a `CASH` account for a backtest venue:

```
from nautilus_trader.adapters.binance import BINANCE_VENUE
from nautilus_trader.backtest.engine import BacktestEngine
from nautilus_trader.model.currencies import USD
from nautilus_trader.model.enums import OmsType, AccountType
from nautilus_trader.model import Money, Currency

# Initialize the backtest engine
engine = BacktestEngine()
```



```
# Add a CASH account for the venue
engine.add_venue(
    venue=BINANCE_VENUE, # Create or reference a Venue identifier
    oms_type=OmsType.NETTING,
    account_type=AccountType.CASH,
    starting_balances=[Money(10_000, USDT)],
)
```

Margin models

Margin models determine how the simulated exchange reserves collateral for orders and positions in backtest runs. The model types (`StandardMarginModel` vs `LeveragedMarginModel`), their formulas, the default behavior, and custom model authoring are covered in the dedicated [Accounting](#) guide.

This section covers only the backtest-specific configuration.

Backtest venue configuration

Specify the margin model on `BacktestVenueConfig` via `MarginModelConfig`:

```
from nautilus_trader.backtest.config import BacktestVenueConfig
from nautilus_trader.backtest.config import MarginModelConfig

venue_config = BacktestVenueConfig(
    name="SIM",
    oms_type="NETTING",
    account_type="MARGIN",
    starting_balances=["1_000_000 USD"],
    margin_model=MarginModelConfig(model_type="standard"), # Options: 'standard', 'leveraged'
)
```

Available `model_type` values:

- `"leveraged"`: margin reduced by leverage (default).
- `"standard"`: fixed percentages (traditional brokers).
- Fully-qualified class path for a custom model: `"my_package.my_module:MyMarginModel"`.

High-level backtest API

When using the high-level API, attach the margin model in the same way:

```
from nautilus_trader.backtest.config import BacktestVenueConfig
from nautilus_trader.backtest.config import MarginModelConfig
from nautilus_trader.config import BacktestRunConfig

venue_config = BacktestVenueConfig(
    name="SIM",
    oms_type="NETTING",
    account_type="MARGIN",
    starting_balances=["1_000_000 USD"],
    margin_model=MarginModelConfig(
        model_type="standard", # Traditional broker simulation
    ),
)

config = BacktestRunConfig(
    venues=[venue_config],
    # ... other config
)
```

Custom model with parameters:

```
margin_model=MarginModelConfig(
    model_type="my_package.my_module:CustomMarginModel",
    config={
        "risk_multiplier": 1.5,
        "use_leverage": False,
        "volatility_threshold": 0.02,
    },
)
```

The model is applied to the simulated exchange during backtest execution.

Trade ID derivation

The simulated exchange (used by both backtest and sandbox execution) emits a deterministic `TradeId` for each generated fill. The ID is formatted as `T-{hash:016x}-{count:03d}`, where the 16-character hex is an FNV-1a hash of `(venue, raw_id, ts_init)` and the trailing counter distinguishes multiple fills at the same `ts_init` (e.g. several legs of a bar-driven fill).

Properties:

- Deterministic across runs: the same replayed data produces the same `TradeId` every time, so downstream dedup and golden-output comparisons stay stable.
- Collision-safe across resets: `ts_init` is pinned in backtest data and monotonic in live/sandbox, so a `BacktestEngine.reset()` (or an in-memory `IdsGenerator` reset in a sandbox with persisted orders) cannot mint a `TradeId` that collides with one already in the cache.
- Bounded length: the hash keeps the identifier under the 36-character `TradeId` cap regardless of venue name length.

The `use_random_ids` venue flag still governs `VenueOrderId` and `PositionId` generation, but `TradeId` is always deterministic and is not affected by the flag.

Related guides

- [Strategies](#) - Develop strategies to backtest.
- [Visualization](#) - Generate tearsheets from backtest results.
- [Reports](#) - Analyze backtest performance data.

[< Logging](#)[Previous Page](#)[Visualization >](#)[Next Page](#)